

P3016R1

Resolve inconsistencies in begin/end for `valarray` and braced initializer lists

Published Proposal, 2023-12-08

Author:

[Arthur O'Dwyer](#)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Draft Revision:

7

Abstract

We show that some inconsistencies between non-member `begin` and `cbegin` applied to `valarray` and to braced initializer lists were caused by C++0x uncertainty around `[stmt.ranged]`, and resolve these inconsistencies for C++26.

Table of Contents

1	Changelog
2	Motivation and proposal
2.1	<code>.data</code> and <code>.empty</code>
3	Historical background
4	Implementation experience
4.1	Tony Table
5	Proposed wording
5.1	<code>[valarray.syn]</code>
5.2	<code>[template.valarray.overview]</code>
5.3	<code>[valarray.members]</code>
5.4	<code>[support.initlist]</code>
5.5	<code>[iterator.synopsis]</code>
5.6	<code>[iterator.range]</code>

5.7 .data+.size cleanup

5.7.1 [string.cons]

5.7.2 [string.append]

5.7.3 [string.assign]

5.7.4 [string.replace]

5.7.5 [span.cons]

5.7.6 [valarray.cons]

6 Proposed straw polls

References

Informative References

§ 1. Changelog

- R1:
 - Split [\[CWG2825\]](#) from this paper; this paper is now library-only.
 - Add discussion of `.data()` and `.empty()` overloads (proposed but undiscussed in R0).
 - Add wording for ".data+.size cleanup."

§ 2. Motivation and proposal

Casey Carter [points out](#) that the following program is supported by libstdc++ but not libc++ nor Microsoft ([Godbolt](#)):

```
#include <iterator>
#include <valarray>
int main() {
    std::valarray<int> v = {1,2,3};
    std::begin(v); // OK
    std::cbegin(v); // Error
}
```

This is because `std::valarray` defines its own non-member, non-hidden-friend overloads of `std::begin` and `std::end`. These overloads are found by the qualified call to `std::begin` here, but aren't found by `std::cbegin`'s ADL because the primary template for `std::cbegin` happens to be defined before `<valarray>` is included. Swapping the order of `#include <iterator>` and `#include <valarray>` in this example doesn't help, because the relevant parts of `<iterator>` are still transitively included by `<valarray>` before `std::valarray`'s own code.

Likewise, on all vendors ([Godbolt](#)):

```
#include <iterator>
int main() {
    std::begin({1,2,3}); // OK
```

```
std::cbegin({1,2,3}); // Error
}
```

This is because `{1,2,3}` is a *braced-initializer-list* with no type; so it cannot bind to the deduced `const C&` in `std::cbegin` (defined in `<iterator>`). But it *can* bind to the `initializer_list<E>` in the non-member, non-hidden-friend overload `std::begin(std::initializer_list<E>)` (defined in `<initializer_list>`).

Notice that `std::begin({1,2,3})` returns an iterator that will dangle at the end of the full-expression, and that the return values of `std::begin({1,2,3})` and `std::end({1,2,3})` do not form a range (because the two lists' backing arrays may be different). Therefore this overload's functionality is more harmful than helpful.

NOTE: Be careful to distinguish the scenario of calling `std::begin` on an *object of type* `initializer_list` (helpful!) from calling it on a *braced-initializer-list* (harmful).

We propose to resolve `valarray`'s `begin/cbegin` inconsistency in favor of "make it work" and to resolve *braced-initializer-list*'s inconsistency in favor of "make it ill-formed."

§ 2.1. `.data` and `.empty`

We also propose two additional member functions for `std::initializer_list`: `.data()` and `.empty()`.

Many places in the library clauses would like to operate on the contiguous data of an `initializer_list` using the "data + size" idiom, but since `initializer_list` lacks `.data()`, they're forced to use an awkward "begin + size" approach instead. This (1) looks unnatural and (2) causes extra mental effort for library writers. Part of P3016's proposed wording is to update these places in the library. For example:

```
constexpr basic_string& append(const basic_string& str);
1. Effects: Equivalent to return append(str.data(), str.size());
[...]
constexpr basic_string& append(initializer_list<charT> il);
16. Effects: Equivalent to return append(il.data(), il.size() il.begin(), il.size());
```

As for `.empty()`, it is generally recognized these days that ranges providing `.size()` should also provide `.empty()`. For example, `iota_view`'s missing `.empty()` was added as a DR by [LWG4001].

By making `il.data()` and `il.empty()` well-formed for `std::initializer_list` objects, we satisfy the SFINAE conditions of the primary templates for `std::data(il)` and `std::empty(il)`, meaning that we can eliminate their special overloads for `initializer_list` arguments.

```
std::initializer_list<int> il = {1,2,3};
const int *p = std::data(il);
// OK, calls the initializer_list overload
// specially provided in <iterator>
bool b = std::empty(il);
// OK, calls the initializer_list overload
// specially provided in <iterator>
```

```
std::initializer_list<int> il = {1,2,3};
const int *p = std::data(il);
// OK, calls the primary template
bool b = std::empty(il);
// OK, calls the primary template
```

```
const int *p = std::data({1,2,3});
// calls the initializer_list overload
// specially provided in <iterator>,
// and immediately dangles
const int *q = std::data({});
// does not compile
```

```
const int *p = std::data({1,2,3});
// does not compile
const int *q = std::data({});
// does not compile
```

```
bool b = std::empty({1,2,3});
// calls the initializer_list overload
// specially provided in <iterator>,
// invariably returns false
bool c = std::empty({});
// does not compile
```

```
bool b = std::empty({1,2,3});
// does not compile
bool c = std::empty({});
// does not compile
```

§ 3. Historical background

[N2930] "Range-Based For Loop Wording (Without Concepts)" (2009) proposed that `[stmt.ranged]` should consider only free function `begin(x)` plus a special case for built-in arrays; therefore N2930 added both `<iterator>`'s `std::begin` and `<initializer_list>`'s `std::begin` — because they wanted `<initializer_list>` not to pull in `<iterator>`, and `initializer_list`'s member function `begin()` wasn't going to be found by their proposed `[stmt.ranged]`.

Then, [N3271] "Wording for Range-Based For Loop (Option #5)" (2011) added the middle bullet point in today's version of `[stmt.ranged]`:

- if `_RangeT` is an array type, `begin-expr` and `end-expr` are `__range` and `__range + __bound`, respectively, where `__bound` is the array bound. If `_RangeT` is an array of unknown size or an array of incomplete type, the program is ill-formed;
- if `_RangeT` is a class type, the *unqualified-ids* `begin` and `end` are looked up in the scope of class `_RangeT` as if by class member access lookup, and if either (or both) finds at least one declaration, `begin-expr` and `end-expr` are `range.begin()` and `range.end()`, respectively;
- otherwise, `begin-expr` and `end-expr` are `begin(__range)` and `end(__range)`, respectively, where `begin` and `end` are looked up with argument-dependent lookup. For the purposes of this name lookup, namespace `std` is an associated namespace.

This change meant that `std::initializer_list` no longer needed to provide free `begin/end`: The new second bullet lets us use `il.begin()` and `il.end()`, so the third bullet is never reached when `decltype(__range)` is `initializer_list`. (Loops of the form `for (int i : {1,2,3})` are subtle because the *for-range-initializer* is a *braced-initializer-list* with no type. [CWG2825] addressed this; GCC, Clang, and EDG already implement the resolution.)

Meanwhile, since the prioritized customization point in `[stmt.ranged]` had shifted from non-member `begin` to member `begin`, it would have made sense for `std::valarray`'s `begin` to change from a non-member to a member. But this was not done — possibly out of C++0x-era concerns that giving `std::valarray` member `begin/end` would make it "too much like a container." [LWG2058] (2011) contains comments like: "The intent of these overloads is entirely to support the new `for` syntax, and not to create new containers." In other words, we wanted `valarray` to be iterable, without looking too much like a new kind of STL container. In 2011, we didn't have a word for that. In 2023, we do: `valarray` is a *range*, just like `array` or `span`, and there's nothing wrong with a *range* having member `begin` and `end`. (P3016 proposes to make it so.)

Similarly, `valarray<T>::iterator` is currently ill-formed even though `ranges::iterator_t<valarray<int>>` is well-formed; `std::rbegin(valarray)` is ill-formed even though `ranges::rbegin(valarray)` is well-formed; and so on.

Finally, [LWG2128] added overloads of `rbegin/rend` for `initializer_list`. The primary template for `rbegin` doesn't work for `initializer_list` because it simply calls `c.rbegin()` — it never attempts to do `make_reverse_iterator(c.end())` because `make_reverse_iterator` isn't SFINAE-friendly. LWG2128's overloads are important because they allow us to write (Godbolt):

```
template<class C>
void f(const C& c) {
    using std::rbegin, std::rend;
    for (auto it = rbegin(c); it != rend(c); ++it) {}
}
void g(std::initializer_list<T> il) {
    f(il);
}
int main() {
    g({1,2,3});
}
```

These were added in `<iterator>`, not in `<initializer_list>`, because they depend on `reverse_iterator`.

However, notice that LWG2128 did *not* add two things:

- `std::cbegin(initializer_list<E>)`. This omission indicates that it was not considered important to support `std::cbegin({1,2,3})`. Unsurprising, as the returned iterator would dangle.
- `std::cbegin(const valarray<T>&)`, `std::rbegin(valarray<T>&)`, `std::rbegin(const valarray<T>&)`, `std::crbegin(const valarray<T>&)`. These omissions might have been accidental; `<valarray>` is easy to overlook. Or, they might indicate the [LWG2058] mindset: `valarray` wasn't intended to be fully rangified, merely iterable with `[stmt.ranged]`, and `[stmt.ranged]` didn't need `cbegin` or `rbegin`.

§ 4. Implementation experience

Arthur has implemented § 5 Proposed wording in his fork of libc++, and used it to compile both LLVM/Clang/libc++ and another large C++17 codebase. Naturally, it caused no problems except in this single test from libc++'s own test suite:

```
#include <initializer_list> // but not <iterator>
std::initializer_list<int> il;
static_assert(noexcept(std::begin(il)));
```

This test now fails first because `<iterator>` was not included, and second because today's `begin(initializer_list<E>)` is noexcept but the primary template `begin(C&)` is non-noexcept (per P0884 guidance).

`il.begin()` and `ranges::begin(il)` remain noexcept.

`std::begin(valarray)` remains non-noexcept.

§ 4.1. Tony Table

<pre>#include <initializer_list> void f(std::initializer_list<int> il) { auto it = std::begin(il); }</pre>	<pre>#include <initializer_list> #include <iterator> // for std::begin void f(std::initializer_list<int> il) { auto it = std::begin(il); }</pre>
<pre>S(std::initializer_list<int> il) : S(il.begin(), il.size()) {}</pre>	<pre>S(std::initializer_list<int> il) : S(il.data(), il.size()) {}</pre>
<pre>auto dangle = std::begin({1,2,3});</pre>	<pre>// no longer compiles</pre>
<pre>bool b = std::empty({1,2,3}); // calls the initializer_list overload // specially provided in <iterator>, // yielding false</pre>	<pre>bool b = std::empty({1,2,3}); // does not compile</pre>
<pre>#include <valarray> #include <utility> std::valarray<int> va; auto it = std::begin(std::as_const(va));</pre>	<pre>#include <valarray> #include <iterator> std::valarray<int> va; auto it = std::cbegin(va);</pre>

§ 5. Proposed wording

§ 5.1. [valarray.syn]

Modify [valarray.syn] as follows:

[...]

```
template<class T> valarray<T> tan (const valarray<T>&);
template<class T> valarray<T> tanh (const valarray<T>&);

template<class T> unspecified1 begin(valarray<T>& v);
template<class T> unspecified2 begin(const valarray<T>& v);
template<class T> unspecified1 end(valarray<T>& v);
template<class T> unspecified2 end(const valarray<T>& v);
}
```

[...]

3. Any function returning a `valarray<T>` is permitted to return an object of another type, provided all the const member functions of `valarray<T>` are also applicable to this type. This return type shall not add more than two levels of template nesting over the most deeply nested argument type.

4. Implementations introducing such replacement types shall provide additional functions and operators as follows:

- (4.1) for every function taking a `const valarray<T>&` ~~other than `begin` and `end`~~, identical functions taking the replacement types shall be added;
- (4.2) for every function taking two `const valarray<T>&` arguments, identical functions taking every combination of `const valarray<T>&` and replacement types shall be added.

5. In particular, an implementation shall allow a `valarray<T>` to be constructed from such replacement types and shall allow assignments and compound assignments of such types to `valarray<T>`, `slice_array<T>`, `gslice_array<T>`, `mask_array<T>` and `indirect_array<T>` objects.

[...]

§ 5.2. [template.valarray.overview]

NOTE: We propose exposition-only `iterator` and `const_iterator` typedefs, but would love to make them non-exposition-only.

NOTE: The `.size()` and `.begin()` members of every STL container are marked `noexcept`. `valarray`'s `.size()` and `std::begin` are non-`noexcept`. We propose `valarray`'s `.begin()` should also be non-`noexcept`, for consistency. If we add any `noexcept` markings to `valarray`, we should add them consistently throughout the class — not *only* on its `begin` and `end`.

Modify [template.valarray.overview] as follows:

```

namespace std {
    template<class T> class valarray {
    public:
        using value_type = T;
        using iterator = unspecified; // exposition only
        using const_iterator = unspecified; // exposition only

        // [valarray.cons], construct/destroy
        valarray();
        explicit valarray(size_t);

    [...]

        // [valarray.range], range access

        iterator begin();
        iterator end();
        const_iterator begin() const;
        const_iterator end() const;

        // [valarray.members], member functions
        void swap(valarray&) noexcept;

        size_t size() const;

        T sum() const;
        T min() const;
        T max() const;

        valarray shift (int) const;
        valarray cshift(int) const;
        valarray apply(T func(T)) const;
        valarray apply(T func(const T&)) const;
        void resize(size_t sz, T c = T());
    };

```

§ 5.3. [valarray.members]

Move the existing section [\[valarray.range\]](#) from its current location to make it a sibling of [\[valarray.members\]](#); then modify it as follows:

~~28.6.10~~ 28.6.2.x **valarray** range access [valarray.range]

1. In the `begin` and `end` function templates that follow, *unspecified1* is a type that **The exposition-only *iterator*** type meets the requirements of a mutable *Cpp17RandomAccessIterator* ([random.access.iterators]) and models `contiguous_iterator` ([iterator.concept.contiguous]), whose `.its` `value_type` is the template parameter `T` and whose `its` reference type is `T&`. *unspecified2* is a type that **The exposition-only *const_iterator*** type meets the requirements of a constant *Cpp17RandomAccessIterator* and models `contiguous_iterator`, whose `.its` `value_type` is the template parameter `T` and whose `its` reference type is `const T&`.
2. The iterators returned by `begin` and `end` for an array are guaranteed to be valid until the member function `resize(size_t, T)` is called for that array or until the lifetime of that array ends, whichever happens first.

```
template<class T> unspecified1 begin(valarray<T>& v);  
template<class T> unspecified2 begin(const valarray<T>& v);  
iterator begin();  
const_iterator begin() const;
```

3. Returns: An iterator referencing the first value in the array.

```
template<class T> unspecified1 end(valarray<T>& v);  
template<class T> unspecified2 end(const valarray<T>& v);  
iterator end();  
const_iterator end() const;
```

4. Returns: An iterator referencing one past the last value in the array.

28.6.2.8 Member functions [valarray.members]

```
void swap(valarray& v) noexcept;
```

1. Effects: `*this` obtains the value of `v`. `v` obtains the value of `*this`.
2. Complexity: Constant.

§ 5.4. [support.initlist]

Modify [support.initlist] as follows:

[...]

17.10.2 Header `<initializer_list>` synopsis [initializer.list.syn]

```
namespace std {
    template<class E> class initializer_list {
    public:
        using value_type      = E;
        using reference        = const E&;
        using const_reference  = const E&;
        using size_type        = size_t;

        using iterator         = const E*;
        using const_iterator   = const E*;

        constexpr initializer_list() noexcept;

        constexpr const E* data() const noexcept;
        constexpr size_t size() const noexcept;    // number of elements
        [[nodiscard]] constexpr bool empty() const noexcept;
        constexpr const E* begin() const noexcept; // first element
        constexpr const E* end() const noexcept;   // one past the last element
    };

    // [support.initlist.range], initializer_list range access
    template<class E> constexpr const E* begin(initializer_list<E> il) noexcept;
    template<class E> constexpr const E* end(initializer_list<E> il) noexcept;
}
```

1. An object of type `initializer_list<E>` provides access to an array of objects of type `const E`.

[Note: A pair of pointers or a pointer plus a length would be obvious representations for `initializer_list`. `initializer_list` is used to implement initializer lists as specified in [dcl.init.list]. Copying an `initializer_list` does not copy the underlying elements. — end note]

2. If an explicit specialization or partial specialization of `initializer_list` is declared, the program is ill-formed.

17.10.3 Initializer list constructors [support.initlist.cons]

```
constexpr initializer_list() noexcept;
```

1. Postconditions: `size() == 0`.

17.10.4 Initializer list access [support.initlist.access]

```
constexpr const E* begin() const noexcept;
```

1. Returns: A pointer to the beginning of the array. If `size() == 0` the values of `begin()` and `end()` are unspecified but they shall be identical.

```
constexpr const E* end() const noexcept;
```

2. Returns: `begin() + size()`.

```
constexpr const E* data() const noexcept;
```

x. Returns: `begin()`.

```
constexpr size_t size() const noexcept;
```

3. Returns: The number of elements in the array.

4. Complexity: Constant ~~time~~.

```
[[nodiscard]] constexpr bool empty() const noexcept;
```

~~x. Returns: `size() == 0`.~~

~~17.10.5 Initializer list range access [support.initlist.range]~~

```
template<class E> constexpr const E* begin(initializer_list<E> il) noexcept;
```

~~1. Returns: `il.begin()`.~~

```
template<class E> constexpr const E* end(initializer_list<E> il) noexcept;
```

~~2. Returns: `il.end()`.~~

§ 5.5. [iterator.synopsis]

Modify [iterator.synopsis] as follows:

25.2 Header synopsis [iterator.synopsis]

```
#include <compare>           // see [compare.syn]
#include <concepts>           // see [concepts.syn]
#include <initializer_list>   // see [initializer.list.syn]

namespace std {

[...]
```

// [iterator.range], range access

```
template<class C> constexpr auto begin(C& c) -> decltype(c.begin());
template<class C> constexpr auto begin(const C& c) -> decltype(c.begin());
template<class C> constexpr auto end(C& c) -> decltype(c.end());
template<class C> constexpr auto end(const C& c) -> decltype(c.end());
template<class T, size_t N> constexpr T* begin(T (&array)[N]) noexcept;
template<class T, size_t N> constexpr T* end(T (&array)[N]) noexcept;
template<class C> constexpr auto cbegin(const C& c)
    noexcept(noexcept(std::begin(c))) -> decltype(std::begin(c));
template<class C> constexpr auto cend(const C& c)
    noexcept(noexcept(std::end(c))) -> decltype(std::end(c));
template<class C> constexpr auto rbegin(C& c) -> decltype(c.rbegin());
template<class C> constexpr auto rbegin(const C& c) -> decltype(c.rbegin());
template<class C> constexpr auto rend(C& c) -> decltype(c.rend());
template<class C> constexpr auto rend(const C& c) -> decltype(c.rend());
template<class T, size_t N> constexpr reverse_iterator<T*> rbegin(T (&array)[N])
template<class T, size_t N> constexpr reverse_iterator<T*> rend(T (&array)[N]);
template<class E> constexpr reverse_iterator<const E*>
    rbegin(initializer_list<E> il);
template<class E> constexpr reverse_iterator<const E*>
    rend(initializer_list<E> il);
template<class C> constexpr auto
    crbegin(const C& c) -> decltype(std::rbegin(c));
template<class C> constexpr auto
    crend(const C& c) -> decltype(std::rend(c));

template<class C> constexpr auto
    size(const C& c) -> decltype(c.size());
template<class T, size_t N> constexpr size_t
    size(const T (&array)[N]) noexcept;

template<class C> constexpr auto
    ssize(const C& c)
        -> common_type_t<ptrdiff_t, make_signed_t<decltype(c.size())>>;
template<class T, ptrdiff_t N> constexpr ptrdiff_t
    ssize(const T (&array)[N]) noexcept;

template<class C> [[nodiscard]] constexpr auto
    empty(const C& c) -> decltype(c.empty());
template<class T, size_t N> [[nodiscard]] constexpr bool
    empty(const T (&array)[N]) noexcept;
template<class E> [[nodiscard]] constexpr bool
    empty(initializer_list<E> il) noexcept;

template<class C> constexpr auto data(C& c) -> decltype(c.data());
template<class C> constexpr auto data(const C& c) -> decltype(c.data());
template<class T, size_t N> constexpr T* data(T (&array)[N]) noexcept;
template<class E> constexpr const E* data(initializer_list<E> il) noexcept;
}
```

§ 5.6. [iterator.range]

Modify [iterator.range] as follows:

```
[...]  
template<class E> [[nodiscard]] constexpr bool empty(initializer_list<E> il) noexcept;  
22. Returns: il.size() == 0;  
[...]  
template<class E> constexpr const E* data(initializer_list<E> il) noexcept;  
25. Returns: il.begin();
```

§ 5.7. .data+.size cleanup

§ 5.7.1. [string.cons]

Modify [string.cons] as follows:

```
constexpr basic_string& operator=(initializer_list<charT> il);  
  
36. Effects: Equivalent to:  
  
return *this = basic_string_view<charT, traits>(il.begin(), il.data(), il.size());
```

§ 5.7.2. [string.append]

Modify [string.append] as follows:

```
constexpr basic_string& append(initializer_list<charT> il);  
  
16. Effects: Equivalent to: return append(il.begin(), il.data(), il.size());
```

§ 5.7.3. [string.assign]

Modify [string.assign] as follows:

```
constexpr basic_string& assign(initializer_list<charT> il);  
  
12. Effects: Equivalent to: return assign(il.begin(), il.data(), il.size());
```

§ 5.7.4. [string.replace]

Modify [string.replace] as follows:

```
constexpr basic_string& replace(const_iterator i1, const_iterator i2, initializer_list<charT> il);
```

12. *Effects*: Equivalent to: return replace(i1, i2, ~~il.begin()~~ il.data(), il.size());

§ 5.7.5. [span.cons]

Modify [span.cons] as follows:

```
constexpr explicit(extent != dynamic_extent) span(std::initializer_list il);
```

18. *Constraints*: is_const_v<element_type> is true.

19. *Preconditions*: If extent is not equal to dynamic_extent, then il.size() is equal to extent.

20. *Effects*: Initializes *data_* with ~~il.begin()~~ il.data() and *size_* with il.size().

§ 5.7.6. [valarray.cons]

Modify [valarray.cons] as follows:

```
valarray(initializer_list<T> il);
```

9. *Effects*: Equivalent to valarray(~~il.begin()~~ il.data(), il.size()).

§ 6. Proposed straw polls

The next revision of this paper (if any) will be guided by the outcomes of these two straw polls.

	<i>SF</i>	<i>F</i>	<i>N</i>	<i>A</i>	<i>SA</i>
Pursue the <i>initializer_list</i> changes and associated library-clause cleanup.	—	—	—	—	—
Pursue the <i>valarray</i> changes.	—	—	—	—	—

§ References

§ Informative References

[CWG2825]

Arthur O'Dwyer. *Range-based for statement using a braced-init-list*. November 2023. URL: <https://cplusplus.github.io/CWG/issues/2825.html>

[LWG2058]

Gabriel Dos Reis. *valarray and begin/end*. May 2011–October 2012. URL: <https://cplusplus.github.io/LWG/issue2058>

[LWG2128]

Dmitry Polukhin. *Absence of global functions cbegin/cend*. January 2012–January 2016. URL: <https://cplusplus.github.io/LWG/issue2128>

[LWG4001]

Hewill Kang. *iota view should provide empty*. October 2023. URL: <https://cplusplus.github.io/LWG/issue4001>

[N2930]

Doug Gregor; Beman Dawes. *Range-Based For Loop Wording (Without Concepts)*. July 2009. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2009/n2930.html>

[N3271]

Doug Gregor. *Wording for Range-Based For Loop (Option #5)*. March 2011. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2011/n3271.htm>